

JavaScript Basics

4-Hour Training Notes • Definitions • Examples • Practical Exercises

Learning Objective

By the end of this session, students should understand JavaScript fundamentals and be able to write small programs using variables, data types, operators, conditions, loops, functions, arrays, and objects.

Hour	Topics	Practical Focus
1	Introduction, HTML integration, console, variables, data types, type conversion, operators	Calculator variable exercises
2	Conditions, ternary, switch, loops, break, continue	Grade, table and number programs
3	Functions, arrow functions, arrays and array methods	Student and marks programs
4	Objects, destructuring, template literals, prompt, revision	Mini student-management program

Hour 1 — JavaScript Fundamentals

1. What is JavaScript?

Definition: JavaScript is a high-level, dynamically typed programming language primarily used to add logic, behavior, and interactivity to web applications.

Example

```
function sayHello() {  
  alert("Hello Students!");  
}
```

2. Adding JavaScript to HTML

Definition: JavaScript can be written inline, inside a script tag, or in an external .js file.

Internal example

```
<script>  
  console.log("Hello JavaScript");  
</script>
```

External example

```
<script src="app.js"></script>
```

3. console.log()

Definition: console.log() displays information in the browser developer console.

Example

```
console.log("Hello World");  
console.log(100);  
console.log(10 + 20);
```

4. Variables

Definition: A variable is a named storage location used to store data that can be accessed and manipulated during program execution.

Example

```
let age = 25;  
age = 26;  
console.log(age);  
  
const pi = 3.14;
```

Modern practice

Prefer let for values that change and const for values that should not be reassigned. var is the older declaration style.

5. Data Types

Definition: A data type defines the kind of value stored in a variable and determines what operations can be performed on that value.

Example

```
let name = "Rahul"; // String  
let age = 21; // Number  
let isStudent = true; // Boolean  
let city; // Undefined  
let selectedUser = null; // Null  
let student = {name: "Rahul", age: 21}; // Object
```

6. typeof

Definition: The typeof operator is used to determine the type of a value.

Example

```
console.log(typeof "Rahul"); // string
console.log(typeof 21); // number
console.log(typeof true); // boolean
```

7. Type Conversion

Definition: Type conversion means converting a value from one data type to another.

Example

```
let age = "25";
let number = Number(age);
console.log(number);

let value = 100;
console.log(String(value));

console.log(parseInt("100"));
console.log(parseFloat("10.50"));
```

8. Operators

Definition: Operators are symbols used to perform operations on values and variables.

Arithmetic

```
let a = 10, b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Comparison

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false
```

Prefer === and !== in modern JavaScript.

Logical

```
let age = 25;
let hasLicense = true;
console.log(age >= 18 && hasLicense);
```

Hour 2 — Conditions and Loops

9. if Statement

Definition: An if statement executes a block of code when a specified condition is true.

Example

```
let age = 20;

if (age >= 18) {
  console.log("Eligible");
}
```

10. if-else

Definition: if-else executes one block when a condition is true and another when it is false.

Example

```
let age = 16;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

11. else-if

Definition: else-if is used when multiple conditions need to be checked.

Example

```
let marks = 75;

if (marks >= 90) {
  console.log("A+");
} else if (marks >= 75) {
  console.log("A");
} else if (marks >= 60) {
  console.log("B");
} else {
  console.log("Fail");
}
```

12. Ternary Operator

Definition: The ternary operator is a short form of a simple if-else expression.

Example

```
let age = 20;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

13. switch

Definition: switch executes different blocks of code based on the value of an expression.

Example

```
let day = 2;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
```

```
    console.log("Tuesday");
    break;
  default:
    console.log("Invalid day");
  }
}
```

14. for Loop

Definition: A for loop is commonly used when the number of iterations is known.

Example

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Multiplication table

```
let number = 5;
for (let i = 1; i <= 10; i++) {
  console.log(number * i);
}
```

15. while Loop

Definition: A while loop executes code repeatedly as long as its condition remains true.

Example

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

16. do...while

Definition: A do...while loop executes the code at least once and then checks the condition.

Example

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);
```

17. break and continue

Definition: break terminates a loop immediately; continue skips the current iteration and moves to the next iteration.

break

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i);
}
```

continue

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);
}
```

Hour 3 — Functions and Arrays

18. Functions

Definition: A function is a reusable block of code designed to perform a specific task.

Example

```
function greet() {  
  console.log("Hello Students");  
}  
  
greet();
```

19. Parameters and Return

Definition: Parameters receive values passed to a function; return sends a result back to the caller.

Example

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(10, 20);  
console.log(result);
```

20. Arrow Functions

Definition: An arrow function is a shorter syntax for writing functions.

Example

```
const add = (a, b) => a + b;  
console.log(add(10, 20));
```

21. Arrays

Definition: An array is a data structure used to store multiple values in a single variable.

Example

```
let fruits = ["Apple", "Mango", "Banana"];  
console.log(fruits[0]);  
console.log(fruits.length);
```

22. Array Methods

Definition: Array methods provide built-in operations for adding, removing, transforming, and searching array elements.

push / pop / unshift / shift

```
let fruits = ["Apple", "Mango"];  
fruits.push("Banana");  
fruits.pop();  
fruits.unshift("Orange");  
fruits.shift();
```

map

```
let numbers = [1, 2, 3, 4];  
let squares = numbers.map(num => num * num);  
console.log(squares);
```

filter

```
let numbers = [10, 15, 20, 25, 30];  
let result = numbers.filter(num => num > 20);  
console.log(result);
```

23. for...of

Definition: for...of iterates directly over the values of an iterable such as an array.

Example

```
let students = ["Rahul", "Amit", "Priya"];

for (let student of students) {
  console.log(student);
}
```

Hour 4 — Objects and Practical JavaScript

24. Objects

Definition: An object is a collection of related data represented using key-value pairs.

Example

```
let student = {  
  name: "Rahul",  
  age: 21,  
  course: "JavaScript"  
};  
  
console.log(student.name);  
console.log(student.age);
```

25. Modify Objects

Definition: Object properties can be updated, added, or deleted.

Example

```
student.age = 22;  
student.city = "Mumbai";  
delete student.city;
```

26. Object Methods and this

Definition: An object method is a function stored as an object property; in a normal method call, this refers to the object making the call.

Example

```
let student = {  
  name: "Rahul",  
  greet: function() {  
    console.log("Hello " + this.name);  
  }  
};  
  
student.greet();
```

27. Destructuring

Definition: Destructuring extracts values from arrays or objects into variables.

Object

```
let student = { name: "Rahul", age: 21 };  
let { name, age } = student;  
console.log(name, age);
```

Array

```
let numbers = [10, 20];  
let [a, b] = numbers;  
console.log(a, b);
```

28. Template Literals

Definition: Template literals create strings using backticks and allow variables to be embedded using `${}`.

Example

```
let name = "Rahul";  
let age = 21;  
console.log(`My name is ${name} and I am ${age} years old.`);
```

29. User Input with prompt()

Definition: prompt() displays a browser input dialog and returns the entered value as a string.

Example

```
let name = prompt("Enter your name");
console.log(`Hello ${name}`);

let age = Number(prompt("Enter your age"));
console.log(age);
```

Practical Exercises

1. **Even or Odd** — Input a number and print whether it is even or odd.
2. **Student Grade** — Accept marks and print a grade using if/else-if.
3. **Multiplication Table** — Accept a number and print its table from 1 to 10.
4. **Shopping Bill** — Accept price and quantity, then calculate total bill.
5. **Largest Number** — Find the largest of three numbers.
6. **Student Array** — Store four student names in an array and print them using for...of.
7. **Passing Students** — Given marks in an array, use filter() to keep marks >= 40.
8. **Student Object** — Create a student object with name, age, marks and course, then display the values.

Topics to Cover in Later JavaScript Sessions

DOM manipulation • Events • Event bubbling • Callbacks • Promises • async/await • Fetch/API • JSON • ES6 modules • Classes • Prototypes • Closures • Hoisting • LocalStorage • Error handling

Trainer Tip: Spend roughly 60% of the session on explanation and live coding and 40% on student hands-on practice. After each concept, ask students to modify the example rather than only copying it.